

LabSoldier 作業指南書 — naganawa (PM + ログイン機能担当)

Hackathon Soldier Team

2026-05-21

目次

はじめに	2
第 1 部 共通の基礎知識	2
1.1 このアプリの全体像	2
1.2 JavaScript / TypeScript の最低限	2
1.3 Node.js / npm の最低限	4
第 2 部 認証の基礎知識	4
2.1 認証 (Authentication) とは	4
2.2 token とは	5
2.3 パスワードの扱い	5
2.4 JWT (JSON Web Token)	5
第 3 部 担当範囲と作成物	6
3.1 担当範囲	6
3.2 完成イメージ	6
第 4 部 実装手順	7
ステップ 0: ブランチを切る	7
ステップ 1: tsutsumi と「user」型をすり合わせる	7
ステップ 2: 必要なライブラリを入れる	7
ステップ 3: ユーザーストアを作る	7
ステップ 4: JWT のヘルパー	9
ステップ 5: auth ルートを本実装	9
ステップ 6: auth middleware を作る	10
ステップ 7: /api/me を本実装	11
ステップ 8: フロントの API クライアントを更新	11
ステップ 9: ログイン画面を改修	12
ステップ 10: サインアップ画面を作る	13

ステップ 11: App.tsx でルーティング	14
ステップ 12: 動作確認	16
ステップ 13: PR を出す	16
第 5 部よくある詰まりどころ	16
bcrypt のインストールでエラー	16
token が undefined になる	16
CORS エラー	16
401 unauthorized が出続ける	16
第 6 部ここまでやったら他の人と合流	17
第 7 部 PM 業務（並行してやる）	17
付録 A: チートシート	17
付録 B: 用語集	18

はじめに

この PDF は、naganawa が ログイン機能（認証フロー全体）を実装するための指南書です。Web 開発の基礎、TypeScript の文法、Node.js / Express、React 認証フローまで一通り解説しています。読みながらコードを書いていけば、ログイン機能が完成します。

第 1 部共通の基礎知識

1.1 このアプリの全体像

[ブラウザ React] ⇄ [Node.js Express] ⇄ [DB (SQLite)]
frontend/ backend/

ローカルでは: - フロント: <http://localhost:5173> - バック: <http://localhost:3001>

1.2 JavaScript / TypeScript の最低限

変数

```
const name = "naganawa";    // 再代入できない
let count = 0;               // 再代入できる
// var は使わない
```

関数

// 通常関数

```
function add(a: number, b: number): number {  
    return a + b;  
}
```

// アロー関数 (よく使う)

```
const add = (a: number, b: number): number => a + b;
```

// 非同期関数

```
async function getUser(id: string): Promise<User> {  
    const res = await fetch(`/api/users/${id}`);  
    return res.json();  
}
```

オブジェクトと配列

```
const user = { id: "u1", name: "naganawa" };  
console.log(user.name);    // "naganawa"
```

```
const users = ["a", "b", "c"];  
users.forEach(u => console.log(u));  
const upper = users.map(u => u.toUpperCase()); // ["A", "B", "C"]  
const found = users.find(u => u === "b");      // "b"
```

型定義

```
interface User {  
    id: string;  
    name: string;  
    passwordHash?: string;    // ?で省略可能  
}
```

```
type Status = "online" | "offline" | "away";    // ユニオン型
```

```
function greet(user: User): string {  
    return `Hello, ${user.name}`;  
}
```

async/await

ネットワーク通信や DB アクセスは「非同期」。await で完了を待つ。

```

async function login(name: string) {
  const res = await fetch("/api/auth/login", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ name }),
  });
  if (!res.ok) {
    throw new Error("login failed");
  }
  const data = await res.json();
  return data;
}

```

try/catch

エラーは try/catch で受ける。

```

try {
  const user = await login("naganawa");
  console.log("ログイン成功", user);
} catch (e) {
  console.error("ログイン失敗", e);
}

```

1.3 Node.js / npm の最低限

- ・ node script.js で JS ファイルを実行
- ・ npm install で package.json のライブラリを入れる
- ・ npm run dev で開発サーバー起動
- ・ import 文で他ファイルを読み込む

```

// 別ファイルから読み込む
import { store } from "../db/store.js";
// ライブラリから読み込む
import express from "express";

```

第 2 部認証の基礎知識

2.1 認証 (Authentication) とは

「あなたは誰？」を確認する仕組み。今回はシンプルに以下を実装。

1. ユーザーは name + password を送る

2. サーバーは DB で照合
3. 一致したらサーバーが **token** を発行
4. クライアントは以後 **token** を持って通信
5. サーバーは **token** を見て「ああ、naganawa さんね」と認識

2.2 token とは

サーバーが「ログインに成功した証」として発行する文字列。

例: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJ1LW5hZ2FuYXdhIn0.xxx

クライアントは localStorage か Cookie に保存し、リクエストの Authorization ヘッダに乗せる。

GET /api/me

Authorization: Bearer eyJhbGc...

2.3 パスワードの扱い

⚠ 絶対にやってはいけないこと: パスワードを平文（そのまま）で DB に保存

正しい方法: bcrypt など で ハッシュ化 してから保存

"hackathon123" → "\$2b\$10\$abcd1234..."

ハッシュは元に戻せない。ログイン時はユーザーが送ってきたパスワードを同じハッシュ関数にかけて、保存値と比較する。

今回は bcrypt ライブラリを使う:

```
cd backend
npm install bcrypt
npm install -D @types/bcrypt

import bcrypt from "bcrypt";

// 登録時
const hash = await bcrypt.hash(rawPassword, 10);

// ログイン時
const ok = await bcrypt.compare(rawPassword, hash);
```

2.4 JWT (JSON Web Token)

今回は JWT を使う。サーバーが秘密鍵で署名した文字列で、改ざんできない。

```
cd backend
npm install jsonwebtoken
```

```

npm install -D @types/jsonwebtoken

import jwt from "jsonwebtoken";

const SECRET = process.env.JWT_SECRET ?? "dev-secret-change-me";

// 発行
const token = jwt.sign({ userId: user.id }, SECRET, { expiresIn: "7d" });

// 検証
try {
  const payload = jwt.verify(token, SECRET) as { userId: string };
  console.log(payload.userId);
} catch (e) {
  // 不正トークン
}

```

第3部担当範囲と作成物

3.1 担当範囲

種類	ファイル	やること
Backend	backend/src/routes/auth.ts	/api/auth/login, /api/auth/signup の本実装
Backend	backend/src/middleware/auth.ts (新規)	tokens 検証ミドルウェア
Backend	backend/src/db/users.ts (新規)	ユーザーテーブル
Backend	backend/src/index.ts	/api/me を auth middleware で保護
Frontend	frontend/src/pages/Login.tsx	パスワード欄追加、エラー表示
Frontend	frontend/src/pages/Signup.tsx (新規)	サインアップ画面
Frontend	frontend/src/api/client.ts	トークン管理を強化
Frontend	frontend/src/App.tsx	ルーティング、ログイン状態の判定

3.2 完成イメージ

1. ユーザーが /signup でアカウントを作る (name + password)

2. ログイン画面で name + password を入れる
 3. ログイン成功 → ホーム画面に遷移
 4. リロードしてもログイン状態が続く (token が localStorage に残ってる)
 5. 7 日経つと token 失効 → ログイン画面に戻る
 6. ログアウトボタンで明示的にログアウト
-

第 4 部 実装手順

ステップ 0: ブランチを切る

```
git switch main
git pull origin main
git switch -c feature/naganawa_auth
```

ステップ 1: tsutsumi と「user」型をすり合わせる

ログイン機能で扱う user の項目を tsutsumi と合わせる。

提案する User テーブル:

```
interface User {
  id: string;           // "u-naganawa" など UUID か手動 ID
  name: string;         // ログイン用、ユニーク
  passwordHash: string; // bcrypt のハッシュ
  avatarId: string;     // "soldier-blue" など
  createdAt: string;    // ISO8601
}
```

tsutsumi の DB スキーマに反映してもらう。

ステップ 2: 必要なライブラリを入れる

```
cd backend
npm install bcrypt jsonwebtoken
npm install -D @types/bcrypt @types/jsonwebtoken
```

ステップ 3: ユーーストアを作る

backend/src/db/users.ts を新規作成:

```
import bcrypt from "bcrypt";
import type { User } from "../types.js";
```

```
// 初期は配列。後で tsutsumi が SQLite に差し替える
const users: (User & { passwordHash: string })[] = [];

export const userStore = {
  async create(name: string, password: string, avatarId: string): Promise<User> {
    if (users.some(u => u.name === name)) {
      throw new Error("name already taken");
    }
    const passwordHash = await bcrypt.hash(password, 10);
    const user = {
      id: `u-${name}`,
      name,
      avatarId,
      passwordHash,
      createdAt: new Date().toISOString(),
    };
    users.push(user);
    const { passwordHash: _, ...rest } = user;
    return rest as User;
  },

  async verify(name: string, password: string): Promise<User | null> {
    const u = users.find(u => u.name === name);
    if (!u) return null;
    const ok = await bcrypt.compare(password, u.passwordHash);
    if (!ok) return null;
    const { passwordHash: _, ...rest } = u;
    return rest as User;
  },

  findById(id: string): User | null {
    const u = users.find(u => u.id === id);
    if (!u) return null;
    const { passwordHash: _, ...rest } = u;
    return rest as User;
  },
};
```

backend/src/types.ts の User に createdAt を追加し、passwordHash を持つ拡張型はストア内部で扱う。

ステップ 4: JWT のヘルパー

backend/src/lib/jwt.ts (新規):

```
import jwt from "jsonwebtoken";

const SECRET = process.env.JWT_SECRET ?? "dev-secret-CHANGE-IN-PROD";

export function signToken(userId: string): string {
  return jwt.sign({ userId }, SECRET, { expiresIn: "7d" });
}

export function verifyToken(token: string): { userId: string } | null {
  try {
    return jwt.verify(token, SECRET) as { userId: string };
  } catch {
    return null;
  }
}
```

ステップ 5: auth ルートを本実装

backend/src/routes/auth.ts を書き換え:

```
import { Router } from "express";
import { userStore } from "../db/users.js";
import { signToken } from "../lib/jwt.js";

export const authRouter = Router();

// POST /api/auth/signup
authRouter.post("/signup", async (req, res) => {
  const { name, password, avatarId } = req.body ?? {};
  if (!name || !password) {
    return res.status(400).json({ error: "name and password required" });
  }
  if (password.length < 6) {
    return res.status(400).json({ error: "password must be 6+ chars" });
  }
  try {
    const user = await userStore.create(name, password, avatarId ?? "soldier-blue");
    const token = signToken(user.id);
```

```

    return res.json({ user, token });
  } catch (e) {
    return res.status(409).json({ error: String(e) });
  }
});

// POST /api/auth/login
authRouter.post("/login", async (req, res) => {
  const { name, password } = req.body ?? {};
  if (!name || !password) {
    return res.status(400).json({ error: "name and password required" });
  }
  const user = await userStore.verify(name, password);
  if (!user) return res.status(401).json({ error: "invalid credentials" });
  const token = signToken(user.id);
  return res.json({ user, token });
});

```

ステップ 6: auth middleware を作る

backend/src/middleware/auth.ts (新規):

```

import type { Request, Response, NextFunction } from "express";
import { verifyToken } from "../lib/jwt.js";
import { userStore } from "../db/users.js";

declare module "express-serve-static-core" {
  interface Request {
    user?: { id: string; name: string; avatarId: string };
  }
}

export function requireAuth(req: Request, res: Response, next: NextFunction) {
  const auth = req.headers.authorization ?? "";
  const token = auth.replace(/^Bearer\s+/, "");
  const payload = verifyToken(token);
  if (!payload) return res.status(401).json({ error: "unauthorized" });
  const user = userStore.findById(payload.userId);
  if (!user) return res.status(401).json({ error: "unauthorized" });
  req.user = user;
  next();
}

```

ステップ 7: /api/me を本実装

backend/src/index.ts を更新:

```
import { requireAuth } from "../middleware/auth.js";

app.get("/api/me", requireAuth, (req, res) => {
  res.json({ user: req.user });
});

presence ルートも requireAuth で保護できる (tsutsumi と相談):

app.use("/api/presence", requireAuth, presenceRouter);
```

ステップ 8: フロントの API クライアントを更新

frontend/src/api/client.ts の login を修正:

```
async login(name: string, password: string): Promise<User> {
  const res = await fetch("/api/auth/login", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ name, password }),
  });
  if (!res.ok) {
    const err = await res.json().catch(() => ({}));
    throw new Error(err.error ?? `login failed: ${res.status}`);
  }
  const data = await res.json();
  this.saveToken(data.token);
  return data.user;
},

async signup(name: string, password: string, avatarId: string): Promise<User> {
  const res = await fetch("/api/auth/signup", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ name, password, avatarId }),
  });
  if (!res.ok) {
    const err = await res.json().catch(() => ({}));
    throw new Error(err.error ?? `signup failed: ${res.status}`);
  }
}
```

```

    const data = await res.json();
    this.saveToken(data.token);
    return data.user;
  },

```

ステップ 9: ログイン画面を改修

frontend/src/pages/Login.tsx でパスワード欄を追加:

```

import { useState } from "react";
import { api } from "../../api/client";
import type { User } from "../../types";

export function Login({ onLogin }: { onLogin: (u: User) => void }) {
  const [name, setName] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState<string | null>(null);
  const [busy, setBusy] = useState(false);

  const submit = async (e: React.FormEvent) => {
    e.preventDefault();
    setBusy(true);
    setError(null);
    try {
      const user = await api.login(name.trim(), password);
      onLogin(user);
    } catch (err) {
      setError(`ログインに失敗しました: ${err}`);
    } finally {
      setBusy(false);
    }
  };

  return (
    <section className="card" style={{ maxWidth: 360, margin: "40px auto" }}>
      <h2 style={{ marginTop: 0 }}>ログイン</h2>
      <form onSubmit={submit}>
        <input
          type="text"
          placeholder="ユーザー名"
          value={name}
          onChange={(e) => setName(e.target.value)}
          autoFocus

```

```

    />
    <input
      type="password"
      placeholder="パスワード"
      value={password}
      onChange={(e) => setPassword(e.target.value)}
      style={{ marginTop: 8 }}
    />
    <button className="primary" disabled={busy || !name || !password} style={{
      {busy ? "...": "ログイン"}
    }}>
  </button>
</form>
{error && <p style={{ color: "crimson" }}>{error}</p>}
<p style={{ marginTop: 16, fontSize: 14 }}>
  アカウントがない? <a href="#" onClick={() => location.hash = "signup"}>サインアップ</a>
</p>
</section>
);
}

```

ステップ 10: サインアップ画面を作る

frontend/src/pages/Signup.tsx (新規):

```

import { useState } from "react";
import { api } from "../api/client";
import type { User } from "../types";

const AVATARS = ["soldier-blue", "soldier-red", "soldier-green", "soldier-yellow"];

export function Signup({ onSignup }: { onSignup: (u: User) => void }) {
  const [name, setName] = useState("");
  const [password, setPassword] = useState("");
  const [avatarId, setAvatarId] = useState(AVATARS[0]);
  const [error, setError] = useState<string | null>(null);
  const [busy, setBusy] = useState(false);

  const submit = async (e: React.FormEvent) => {
    e.preventDefault();
    setBusy(true);
    setError(null);
    try {
      const user = await api.signup(name.trim(), password, avatarId);
    }
  }
}

```

```

    onSignup(user);
  } catch (err) {
    setError(String(err));
  } finally {
    setBusy(false);
  }
};

return (
  <section className="card" style={{ maxWidth: 360, margin: "40px auto" }}>
    <h2 style={{ marginTop: 0 }}>サインアップ</h2>
    <form onSubmit={submit}>
      <input type="text" placeholder="ユーザー名" value={name} onChange={(e) => set
      <input type="password" placeholder="パスワード (6 文字以上) " value={password} o
      <select value={avatarId} onChange={(e) => setAvatarId(e.target.value)} sty
        {AVATARS.map(a => <option key={a} value={a}>{a}</option>)}
      </select>
      <button className="primary" disabled={busy} style={{ marginTop: 12 }}>
        {busy ? "... " : "登録"}
      </button>
    </form>
    {error && <p style={{ color: "crimson" }}>{error}</p>}
  </section>
);
}

```

ステップ 11: App.tsx でルーティング

```

import { useEffect, useState } from "react";
import { api } from "../api/client";
import { usePresencePing } from "../hooks/usePresencePing";
import { PresenceList } from "../components/PresenceList";
import { ManualCheckin } from "../components/ManualCheckin";
import { Login } from "../pages/Login";
import { Signup } from "../pages/Signup";
import type { User } from "../types";

function App() {
  const [me, setMe] = useState<User | null>(null);
  const [loading, setLoading] = useState(true);
  const [page, setPage] = useState<"login" | "signup">("login");

```

```

useEffect(() => {
  api.me().then((u) => setMe(u)).finally(() => setLoading(false));
  const handler = () => setPage(location.hash === "#signup" ? "signup" : "login");
  window.addEventListener("hashchange", handler);
  handler();
  return () => window.removeEventListener("hashchange", handler);
}, []);

usePresencePing(me !== null);

const logout = () => {
  api.clearToken();
  setMe(null);
};

if (loading) return <p>読み込み中...</p>;
if (!me) {
  return page === "signup"
    ? <Signup onSignup={setMe} />
    : <Login onLogin={setMe} />;
}

return (
  <>
    <header>
      <div>
        <h1>□ LabSoldier</h1>
        <small>研究室にいる仲間が見える化</small>
      </div>
      <div className="row">
        <small>{me.name}</small>
        <button onClick={logout}>ログアウト</button>
      </div>
    </header>
    <PresenceList />
    <ManualCheckin />
  </>
);
}

export default App;

```

ステップ 12: 動作確認

1. backend と frontend を起動
2. ブラウザで `http://localhost:5173` を開く
3. サインアップ画面に行く (リンクから)
4. `name=naganawa, password=hackathon, avatar` 選択 → 登録
5. 自動でホーム画面に遷移、キャラが見える
6. リロード → ログイン状態が維持されている
7. ログアウト → ログイン画面に戻る
8. 同じ `name/password` でログイン → 入れる
9. 違うパスワードでログイン → エラー表示

ステップ 13: PR を出す

```
git add backend/src/ frontend/src/  
git commit -m "feat: 認証機能を実装 (signup/login/me) "  
git push -u origin feature/naganawa_auth
```

GitHub で PR を作成 → tsutsumi と takebayashi にレビュー依頼。

第 5 部よくある詰まりどころ

bcrypt のインストールでエラー

```
xcode-select --install  
rm -rf node_modules  
npm install
```

token が undefined になる

- ・ フロントで `localStorage.setItem` できているか devtools で確認
- ・ リクエスト時に `Authorization` ヘッダがついているか Network タブで確認

CORS エラー

backend/src/index.ts で `app.use(cors());` が入っているか確認。

401 unauthorized が出続ける

- ・ token が古い可能性。 `localStorage.clear()` して再ログイン

- ・ JWT_SECRET を変更したらすべての token が無効になる

第6部ここまでやったら他の人と合流

マイルストーン	誰と何をすり合わせる
ステップ1完了	tsutsumi: user 型を確定
ステップ7完了	tsutsumi: presence ルートにも requireAuth を入れるか相談
ステップ12完了	takebayashi: ログイン → ホーム画面の動線をチェック
PR マージ後	全員: 動作確認

第7部 PM 業務（並行してやる）

- ☐ Day1: 研究室 Wi-Fi のグローバル IP を実測 (<https://api.ipify.org>) し、backend/.env に反映
- ☐ PR レビュー（毎日チェック）
- ☐ 詰まってる人を見つけたらヘルプ
- ☐ デプロイ先選定（Vercel + Render など）
- ☐ 5/27 中間チェック会の進行
- ☐ 6/2 研究室実機テストの段取り
- ☐ 発表スライド作成（6/4-5）

付録 A: チートシート

よく使うコマンド

```
cd backend && npm run dev
cd frontend && npm run dev
git status
git diff
git add <ファイル>
git commit -m "feat: ..."
git push
```

```
# バックエンド起動
# フロントエンド起動
# 何が変わったか
# 中身の差分
# ステージング
# コミット
# GitHub に送る
```

トラブル時

```
rm -rf node_modules && npm install
lsof -i :3001 && kill <PID>
```

```
# 依存をやり直す
# ポート競合解消
```

付録 B: 用語集

- ・ **Hash:** パスワードを「戻せない暗号」に変換する操作
- ・ **bcrypt:** 有名なハッシュライブラリ
- ・ **JWT:** JSON Web Token。署名付きの token
- ・ **Bearer Token:** HTTP の Authorization ヘッダで使う形式
- ・ **Middleware:** リクエストを処理する前後に走る関数
- ・ **CORS:** 別オリジン（ドメイン）からのアクセスを許可する仕組み
- ・ **localStorage:** ブラウザにデータを保存できる仕組み

この指南書を一通り終わると、ログイン機能が完成します。困ったらすぐ LINE で相談。